Day 11 – React Hooks & API Integration

Beginner-friendly notes covering React Hooks, useEffect(), APIs, async/await, loading and error states, cleanup, Local Storage, and the User Management Application.

1. React Hooks

Hooks are special React functions that give functional components extra abilities.

```
useState()   → stores and updates changing data
useEffect()  → performs side effects after rendering
```

2. useState() – Revision

useState() lets a component store data that can change. It returns the current value and a function used to update it.

```
const [name, setName] = useState("Namratha")

name       → current state value
setName    → function used to update the state
```

When state is updated, React re-renders the component so the UI can show the new state.

3. Render and Re-render

Rendering means React takes the component code and determines what should appear on the screen. Re-rendering means React does that again after a relevant change, such as a state update. It does not mean refreshing the whole browser page.

```
setCount(5)
    ↓
state changes
    ↓
React re-renders
    ↓
UI updates
```

4. What is useEffect()?

useEffect() is used when a component needs to perform a side effect after rendering. Common examples are API requests, timers, event listeners, and browser storage operations.

```
import { useEffect } from "react"

useEffect(() => {
  console.log("Hello from useEffect")
}, [])
```

5. Dependency Array

The second argument of useEffect() is the dependency array. It tells React when the effect should run.

Empty dependency array

```
useEffect(() => {
  console.log("Component loaded")
}, [])
```

An empty array means the effect runs when the component mounts, meaning it appears for the first time.

Specific dependency

```
useEffect(() => {
  console.log("Count changed")
}, [count])
```

The effect runs after rendering when count changes.

No dependency array

```
useEffect(() => {
  console.log("Rendered")
})
```

Without a dependency array, the effect runs after every render. This should not normally be used for API fetching.

6. Component Lifecycle

For beginner understanding: Mount = component appears, Update = state or props change and the component renders again, Unmount = component is removed.

```
Mount → component appears
Update → state/props change
Unmount → component disappears
```

useEffect() can run after mounting or after updates depending on its dependency array. Its cleanup function runs when the effect is cleaned up.

7. APIs and fetch()

An API allows an application to request data or services from another system. We will use the JSONPlaceholder public users endpoint.

```
https://jsonplaceholder.typicode.com/users
```

The response contains information such as id, name, username, email, address, phone, website, and company.

8. async / await

Network requests take time. async/await makes asynchronous JavaScript easier to read.

```
async function getUsers() {
  const response = await fetch(
    "https://jsonplaceholder.typicode.com/users"
  )

  const data = await response.json()

  console.log(data)
}
```

fetch() requests the resource. response.json() converts the response body into JavaScript data. await waits for each asynchronous operation before continuing.

9. Fetching API Data in React

```
const [users, setUsers] = useState([])
```

```
useEffect(() => {
  // fetch users here
}, [])
```

Typical flow: component mounts → useEffect runs → API request → data received → setUsers(data) → React re-renders → users appear.

```
Component loads
      ↓
useEffect()
      ↓
fetch API
      ↓
receive data
      ↓
setUsers(data)
      ↓
React re-renders
      ↓
Display users
```

10. Loading State

API requests take time, so a loading state can tell the user that data is being fetched.

```
const [loading, setLoading] = useState(true)

setLoading(false)
```

```
{loading ? (
  <p>Loading users...</p>
) : (
  <UserList users={users} />
)}
```

11. Error Handling

Requests can fail because of network problems or server errors. Store an error message in state and display it conditionally.

```
const [error, setError] = useState("")

setError("Failed to fetch users")
```

A common approach is to use try/catch around the asynchronous request.

12. Cleanup Function

Some effects create resources such as timers or event listeners. These should be cleaned up when they are no longer needed.

```
useEffect(() => {
  const timer = setInterval(() => {
    console.log("Running")
  }, 1000)

  return () => {
    clearInterval(timer)
  }
}, [])
```

The function returned from useEffect() is the cleanup function.

13. Local Storage with React

Local Storage saves small pieces of data in the browser so they can remain available after a page refresh.

```
localStorage.setItem("name", "Namratha")

const name = localStorage.getItem("name")
```

In this project, Local Storage will be used to remember favourite users.

14. Separating API Logic from UI

API code can be placed in a services folder instead of putting everything inside App.jsx. This keeps API logic separate from UI components.

```
src/
■■■ components/
■   ■■■ UserList.jsx
■   ■■■ UserCard.jsx
■   ■■■ SearchBar.jsx
■   ■■■ Loading.jsx
■   ■■■ ErrorMessage.jsx
■■■ services/
■   ■■■ userService.js
■■■ App.jsx
■■■ main.jsx
■■■ index.css
```

15. Day 11 Practical – User Management Application

The application fetches users from a public API, displays reusable user cards, supports search, shows user details, and lets the user add or remove favourite users.

• Fetch users from a public API.

• Display users in card/list format.

• Use useState() to manage user data.

• Use useEffect() to fetch data when the component loads.

• Display a loading indicator while fetching.

• Display an error message if the request fails.

• Search users by name or email.

• Display a selected user's details.

• Store favourite users in Local Storage.

• Add/remove favourites.

• Use reusable components.

16. Application Data Flow

```
App
■
■■■ SearchBar
```

```
■■■ Loading
■■■ ErrorMessage
■■■ UserList
      ■■■ UserCard

App state
   ↓
useEffect()
   ↓
API request
   ↓
userService.js
   ↓
users data
   ↓
setUsers()
   ↓
UserList
   ↓
UserCard
```

17. Important Code Patterns

```
// State
const [users, setUsers] = useState([])

// Effect on mount
useEffect(() => {
  // code
}, [])

// Effect when count changes
useEffect(() => {
  // code
}, [count])

// Fetch
const response = await fetch(url)

// Convert response to JSON
const data = await response.json()

// Store API data
setUsers(data)

// Loading
const [loading, setLoading] = useState(true)

// Error
const [error, setError] = useState("")
```

18. Quick Revision

```
useState()    → stores changing data
useEffect()   → performs side effects
[]            → effect on component mount
[count]       → effect when count changes
fetch()       → requests API data
async/await   → handles asynchronous operations
loading       → tells user data is being fetched
error         → handles failed requests
```

```
cleanup       → removes timers/listeners/resources
localStorage  → saves browser data
```

19. Main Concept

```
useState
   ↓
Store users

useEffect
   ↓
Fetch users when component loads

fetch()
   ↓
API request

setUsers()
   ↓
Update React state

Re-render
   ↓
Display updated users
```

Remember: State changes → React re-renders → UI updates. Rendering means React determines what the component should display; re-rendering means it does that again after a relevant change.